



MPL Assignment 02.

- 1) Define progressive webapp (PWA) and explain its significance in modern web development. Discuss the key characteristic PWAs from traditional mobile Apps.

Solⁿ:

A progressive webapp (PWA) is a type of web application that works like a mobile app but runs in a browser.

Significance of PWA in modern web development:

- (1) Cross platform compatibility.
- (2) Offline support
- (3) Fast performance
- (4) NO app store required.
- (5) Lower development cost

Difference in PWA & Traditional Apps:

Features	PWA	Traditional App
(1) Installation	Direct from Browser	Download from App store.
(2) Internet Required	work offline with caching	usually requires internet
(3) Performance	Fast with service workers	fast but need installation
(4) Updates	Automatic	Manual
(5) Development cost	Lower	Higher.

Q2 Define responsive web design and explain its importance in the context of progressive web. Compare and contrast responsive, fluid and a web design approaches.

Soln:

Definition of Responsive web design:

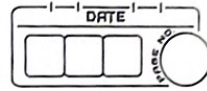
Responsive web design (RWD) is a technique that makes web pages adjust automatically different screen size and devices it ensures a good user experience on mobile, tablets and desktops without needing ~~renew~~ separate version of website.

Importance of Responsive Design in PWA's

- 1) Better user experience.
- 2) Faster load time.
- 3) SEO benefits.
- 4) Cost effective.

Comparisons:

Approach	How it works	Pros	Cons.
Responsive	are flexible grids and CSS media queries.	works on all devices	Can be Comp
Fluid.	are variable widths instead of fixed pixels.	work well on different screen size.	less control over



Key difference:

- 1) Responsive: adapt dynamically to all screens
- 2) Fluid: resizes smoothly but may not be fully optimized.
- 3) Adaptive: loads different layout based on device type.

Describe the lifecycle of service workers, including registration, installation and activation phases.

Soln:

Lifecycle of service workers:

A service worker is a script that runs in the background and helps a web app work offline, load faster and send push notifications.

its lifecycle has three main phases-

1) Registration phase:

- the browser registers the service worker using javascript.

Eg: (service worker in navigator) {

if (navigator.serviceWorker.register("sw.js"))
- then () + console.log("Service Registered")
- catch (error) => console.log("Registration failed");
}

2 Installation phase:

i) the service worker downloads necessary (HTML, CSS, JS) and stores them in cache

ii if successful, it moves to the activation code eg:

```
self.addEventListener('install', event =>
```

```
event.waitUntil(
```

```
  (cache.open({app: 'Cache'}) then
```

```
    return cache.addAll(['/', 'index.html',
```

```
    ],
```

```
  );
```

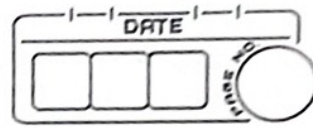
```
});
```

3 Activation phase

- The old service worker is replaced with new
- unused cache files from the previous version are deleted.

~~Final Step: Fetch & Sync.~~

once activated, the service worker intercepts network request serves cached files and syncs data.



Explain the use of indexed DB in the service worker for data storage.

Soln:

use of IndexedDB in service worker for Data storage:

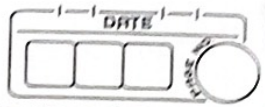
Indexed DB. is a Browser data base that stores large amount of structured data like JSON Objects.

It helps PWs work offline by saving and retrieving data efficiently.

Why are IndexedDB in service workers?

- 1 Offline Support - store data when offline and sync it later
- 2 Efficient storage - saves structured data like user setting cart items as form input
- 3 Faster Access - Retrieves Data quickly without needing a network request
- 4 persistent Data - Data remains saved even after the browser is closed

How service workers are IndexedDB?



opening the Database.

let db:

```
let request: indexed DB open (my database  
request on success = function (event)  
  db = event.target.result;  
};
```

Creating a store & adding Data

```
request on upgrade needed = function (event)  
  let db = event.target.result;  
  let store = db.createObjectStore  
  (user & key path: 'id');  
  store.add (sid: 1, name: 'John Doe',  
  age: 25);  
};
```

Fetching Data in Service Worker

```
let transaction = db.transaction ('user  
read only'); let store =  
transaction.objectStore ('user');
```

```
let get User = store.get (1);  
get user on success = function () {  
  console.log (get user result);  
};
```

~~✗~~